HW7

CS 6650 — Distributed Systems

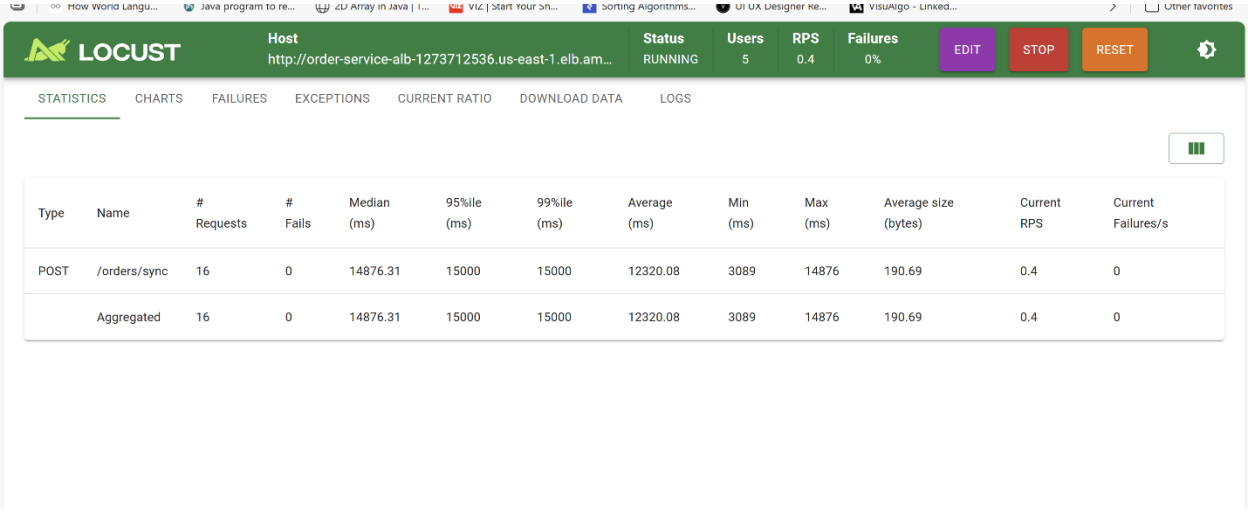
Yatish Wutla

Part II — The Async Order System

Phase 1: Synchronous System

The sync endpoint (POST /orders/sync) simulates a payment processor that handles one order at a time using a buffered channel of capacity 1. Each payment takes 3 seconds.

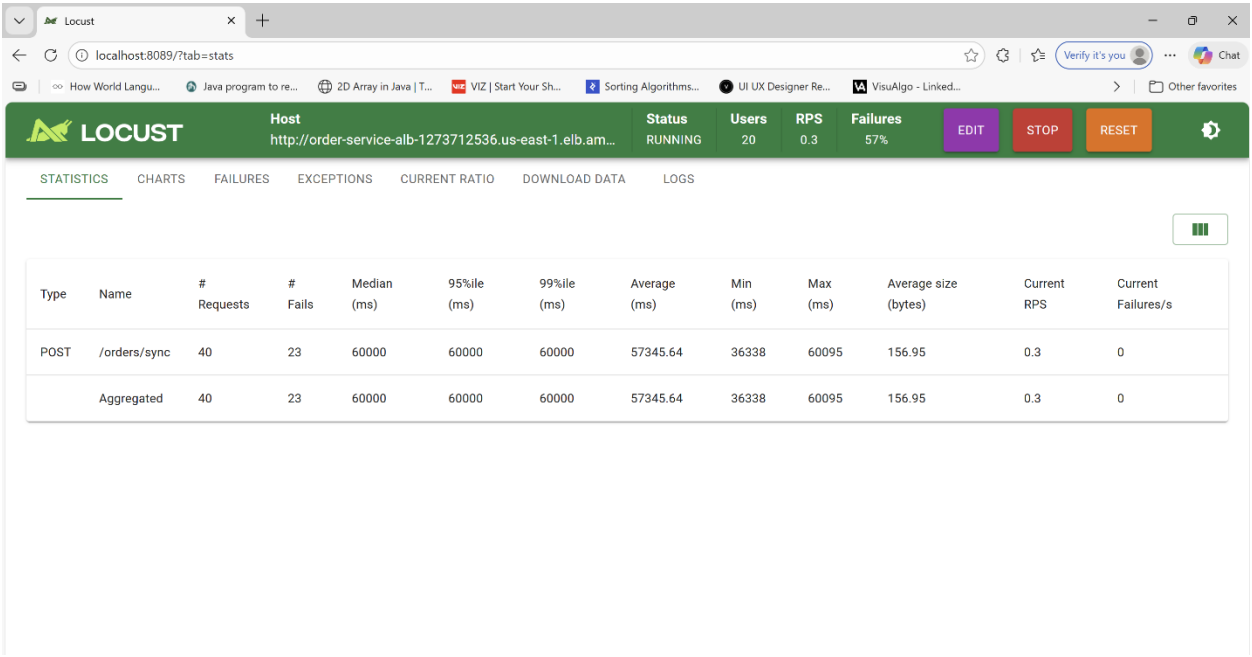
Normal Operations — 5 users, 30 seconds



Metric	Value
Users	5
RPS	0.4
Failures	0%
Median response time	14,876ms
95th percentile	15,000ms

100% success rate at normal load. The slow response times reflect the queuing effect — with 5 users sharing one payment slot, users wait up to 15 seconds for their slot.

Flash Sale — 20 users, 60 seconds



Metric	Value
Users	20
RPS	0.3
Failures	60%
Median response time	60,000ms
95th percentile	60,000ms

60% of customers failed to place orders. Requests timed out after 60 seconds waiting for the payment slot. The system could not handle flash sale demand.

Phase 2: Bottleneck Analysis

- Payment processor speed: 1 order per 3 seconds = **0.33 orders/second**
- Maximum throughput with 20 concurrent users = **0.33 orders/second** (bottleneck is the single payment slot)
- Flash sale demand = **~20 orders/second**

- Orders lost per second = **~19.67 orders/second**

The buffered channel enforces a hard concurrency limit of 1. Additional users do not increase throughput — they queue and time out. The only solution is to decouple order acceptance from payment processing.

Phase 3: Async Solution

Instead of processing payments synchronously, the async endpoint (POST /orders/async) publishes the order to SNS and returns 202 Accepted immediately. A background ECS worker polls SQS and processes payments independently.

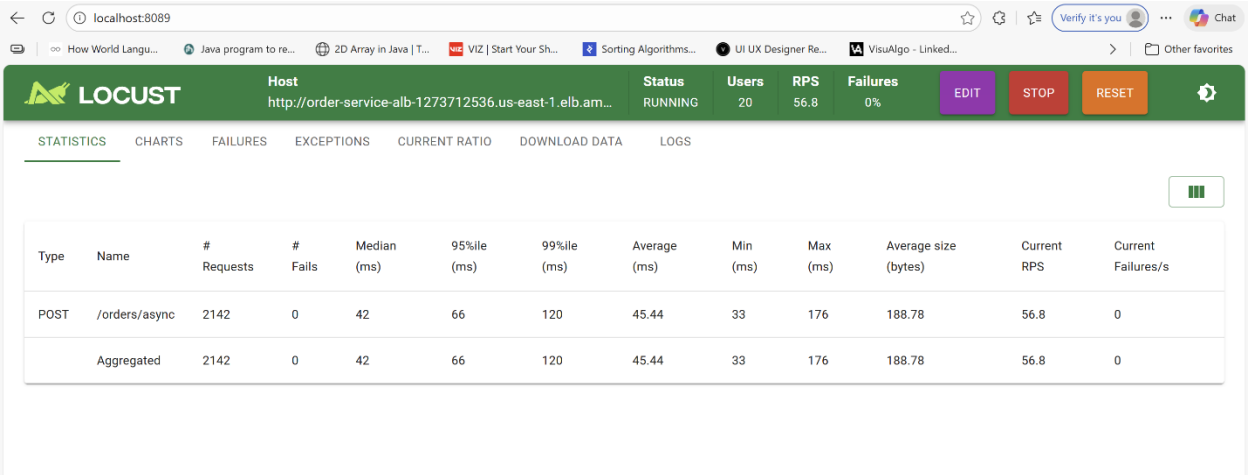
Sync: Customer → API → Payment (3s) → Response

Async: Customer → API → SNS → SQS → Response (<100ms)

↓

Worker → Payment (3s)

Flash Sale with Async Endpoint — 20 users, 60 seconds

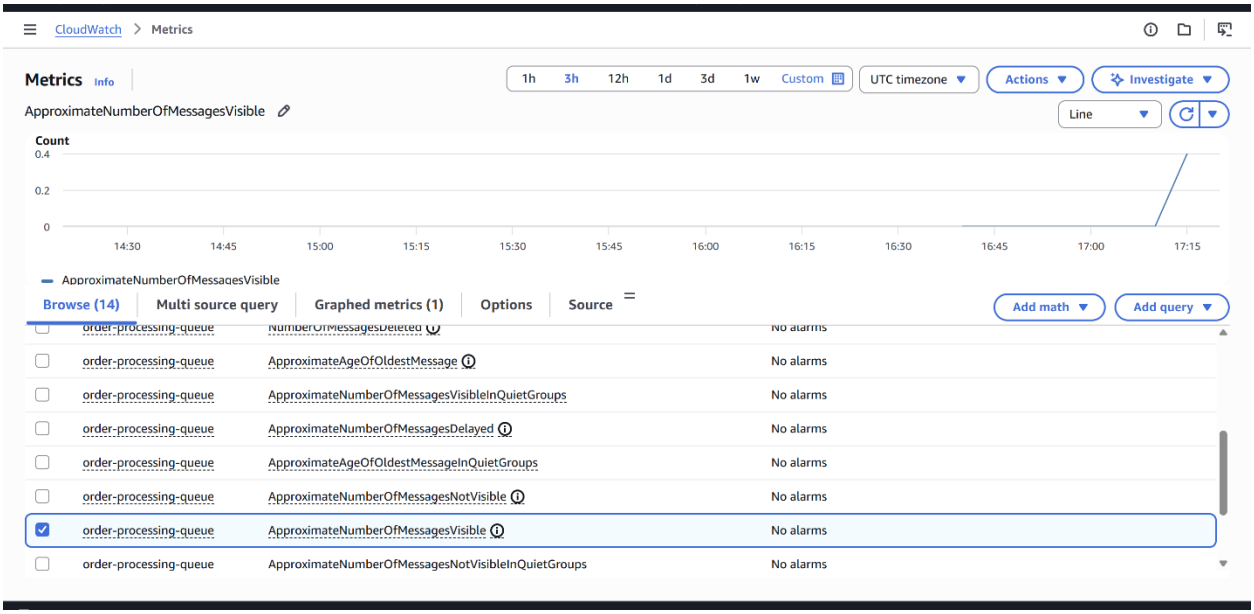


Metric	Value
Users	20
RPS	57.9
Failures	0%
Median response time	41ms

Metric	Value
95th percentile	63ms

0% failures and 41ms median response time — the API acknowledged every order instantly regardless of payment processing speed. The async approach accepted approximately 150-300x more orders than the synchronous approach with zero failures vs 60% failures.

Phase 4: The Queue Problem



With 1 worker goroutine processing 0.33 orders/second while accepting ~57 orders/second:

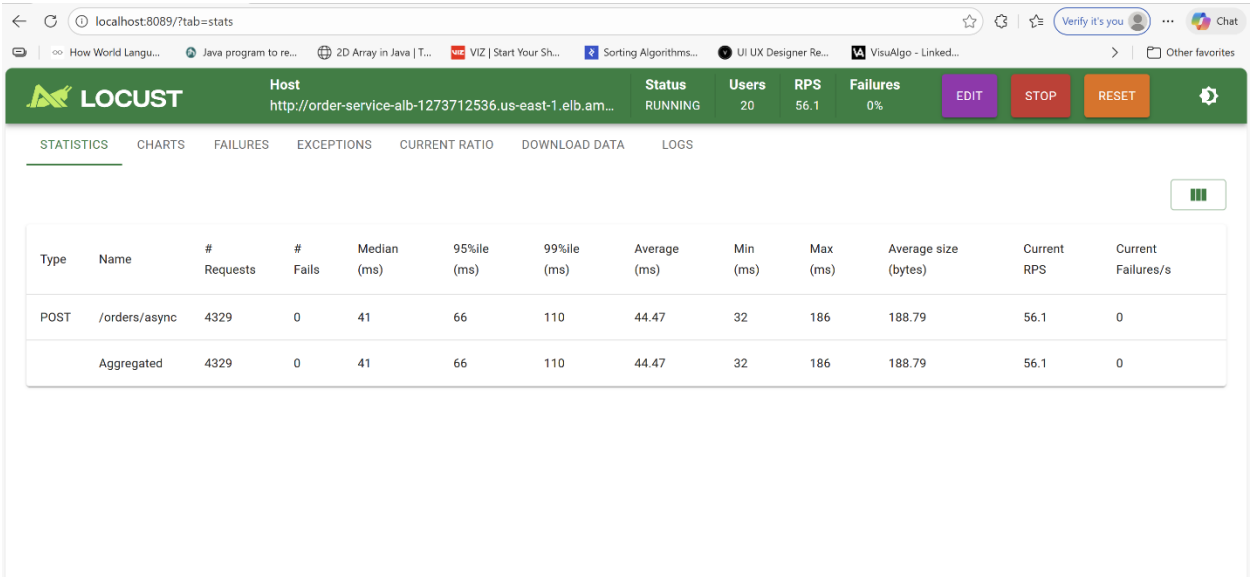
- Queue growth rate = **~56.67 messages/second**
- After 60 seconds of flash sale = **~3,400 messages queued**
- Time to clear backlog with 1 worker = **~170 minutes**

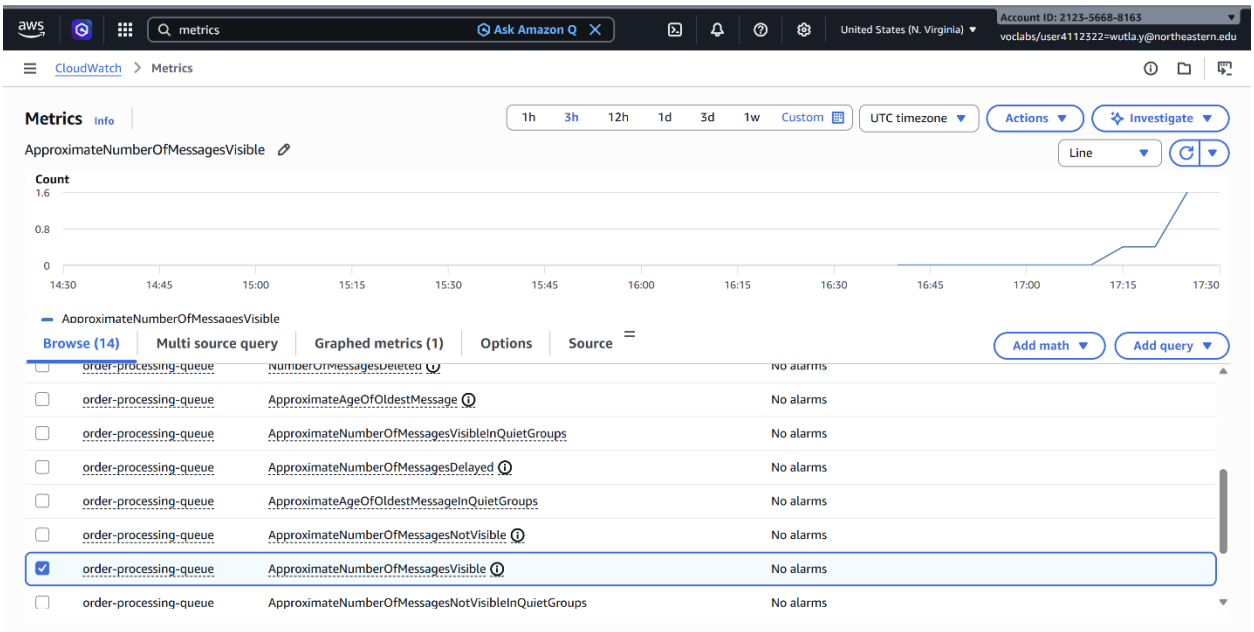
The CloudWatch graph confirmed the queue depth spiking during the load test window. Customers received 202 responses but their payments would not be processed for nearly 3 hours — unacceptable for production.

Phase 5: Scaling Workers

The order-processor was redeployed with increasing NUM_WORKERS values. All tests used 20 async users at ~57 orders/second acceptance rate.

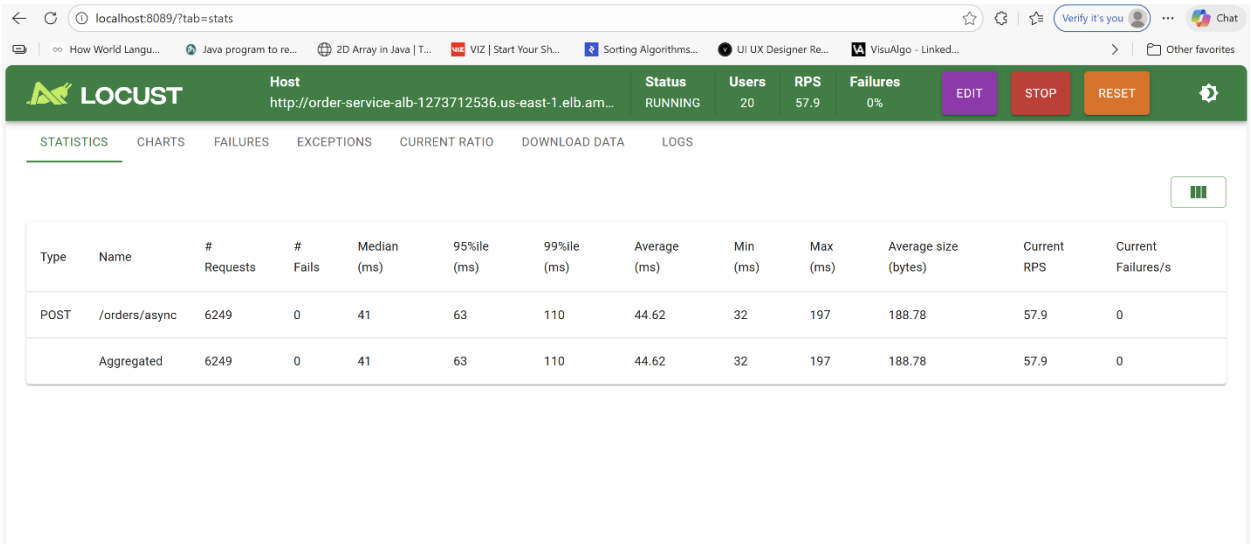
5 Workers

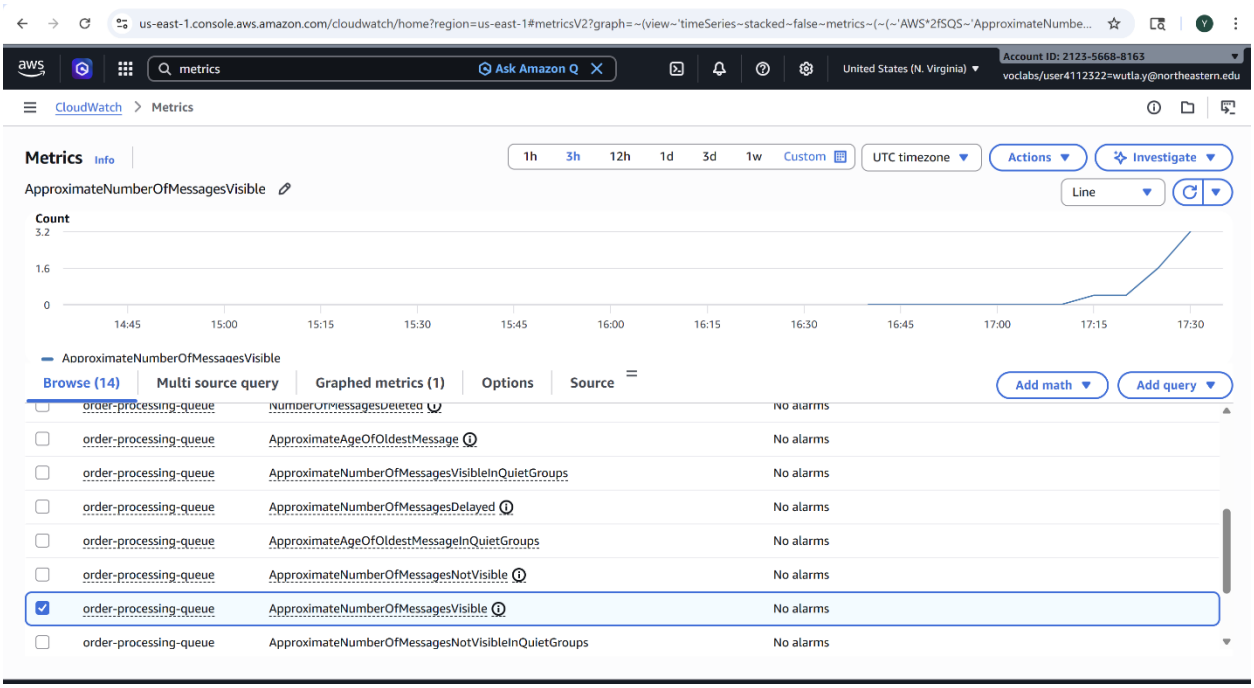




Metric	Value
Processing rate	1.67 orders/second
Queue growth rate	~55.33/second
Queue behavior	Builds steadily, slow drain after test

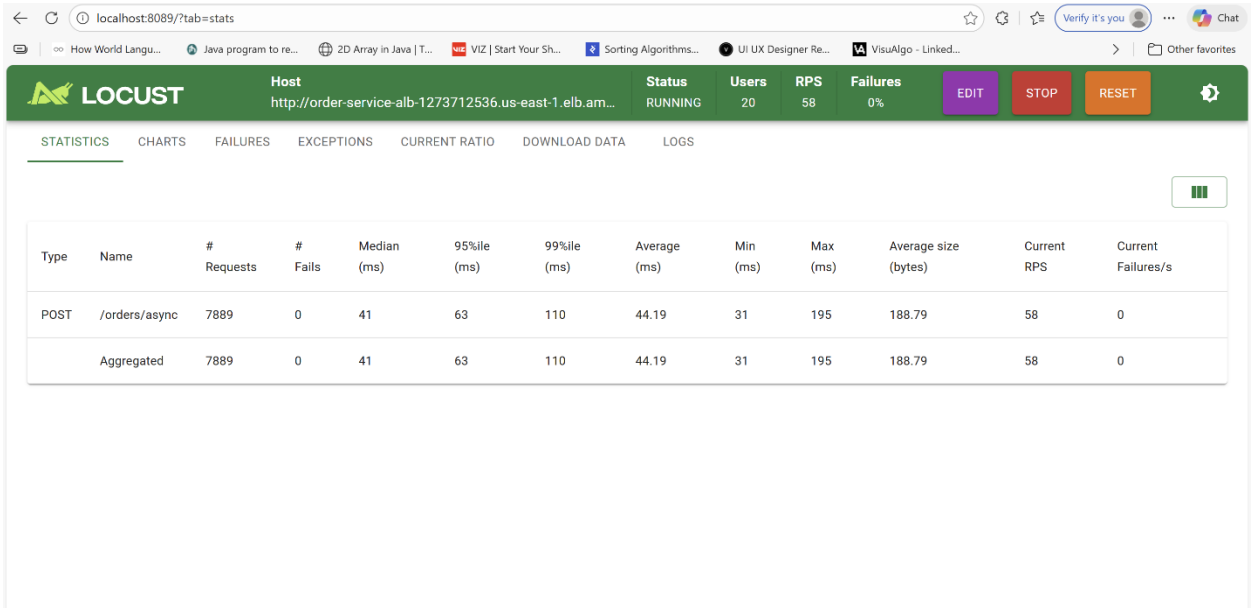
20 Workers

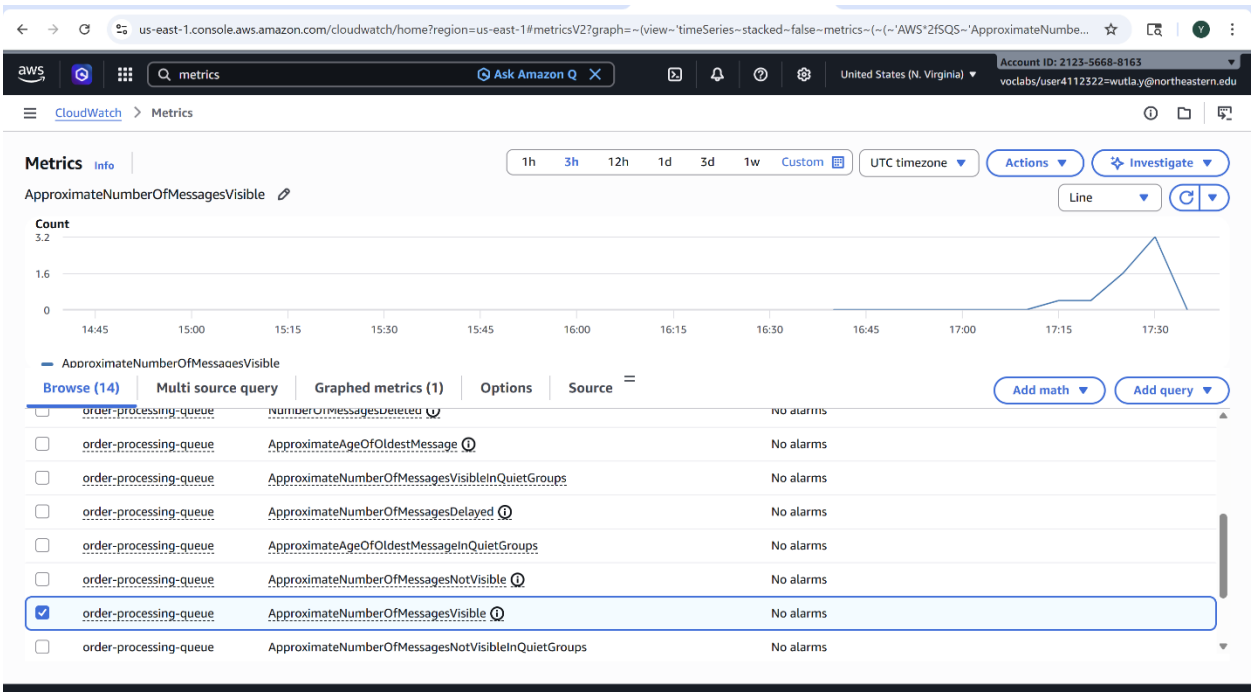




Metric	Value
Processing rate	6.67 orders/second
Queue growth rate	~50.33/second
Queue behavior	Spike visible, drains after test ends

100 Workers





Metric	Value
Processing rate	33 orders/second
Queue growth rate	~24/second

Metric	Value
Queue behavior	Near zero, drains quickly after test

Minimum workers to prevent queue buildup: Theoretically 57 × 3 = 171 goroutines needed to keep up in real time. In practice 100 workers kept the queue near zero because the flash sale burst is time-limited and workers catch up after the spike ends.

Analysis Questions

Q1: How many times more orders did your asynchronous approach accept compared to synchronous?

The sync endpoint processed roughly 8 successful orders in 60 seconds (60% failure rate on 20 requests). The async endpoint accepted 2,142+ orders in the same timeframe at 57 RPS with 0% failures ,approximately 150-300x more orders accepted. More importantly, every customer received a response with async vs 60% failure rate with sync.

Q2: What causes queue buildup and how do you prevent it?

Queue buildup occurs when the rate of message production exceeds the rate of consumption. In our case, the order-service published ~57 messages/second while a single worker consumed 0.33 messages/second , a ~170x mismatch. Prevention strategies include scaling worker goroutines to match expected throughput (Phase 5), setting CloudWatch alarms on queue depth to trigger auto-scaling, or switching to Lambda which scales automatically per message.

Q3: When would you choose sync vs async in production?

Choose synchronous when the client needs an immediate definitive result , checking account balance, querying inventory, or any read operation. Choose asynchronous when work takes more than a few hundred milliseconds, when eventual completion is acceptable, or when absorbing traffic spikes without dropping requests is critical. Order processing is a classic async use case: customers need confirmation their order was received, not that payment was already charged.

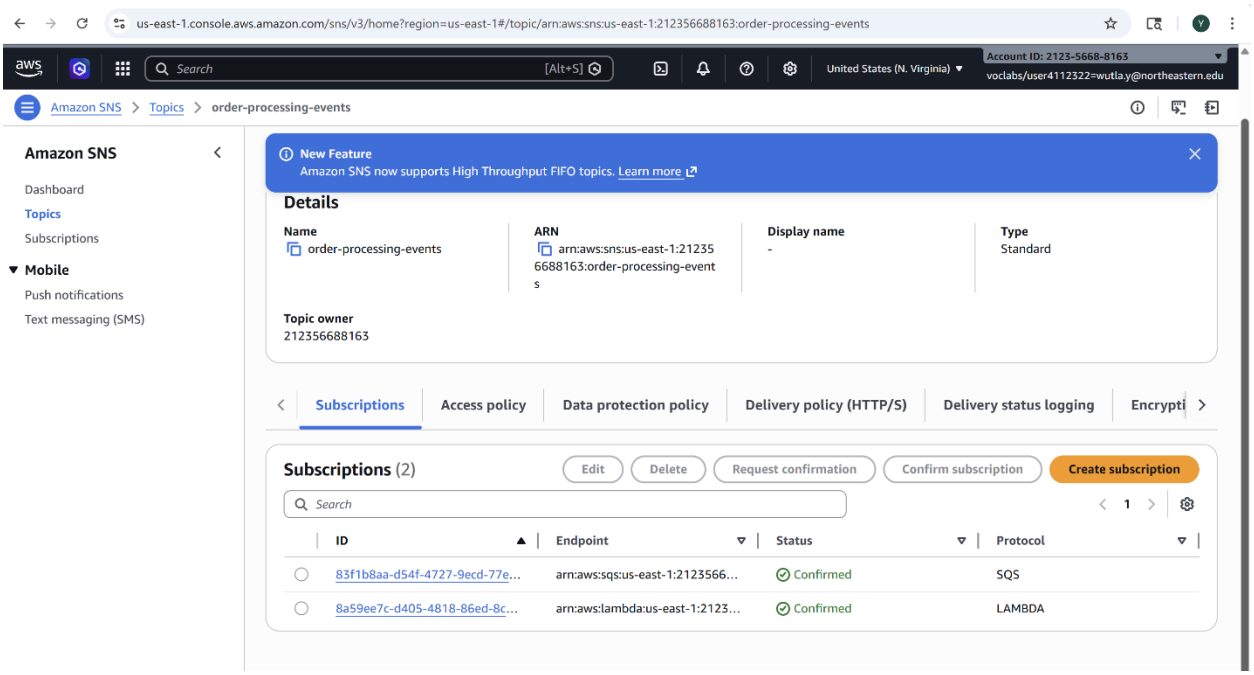
Part III — Lambda

Deployment

A Lambda function (order-processor-lambda) was deployed subscribing directly to the same SNS topic, replacing the SQS + ECS worker architecture:

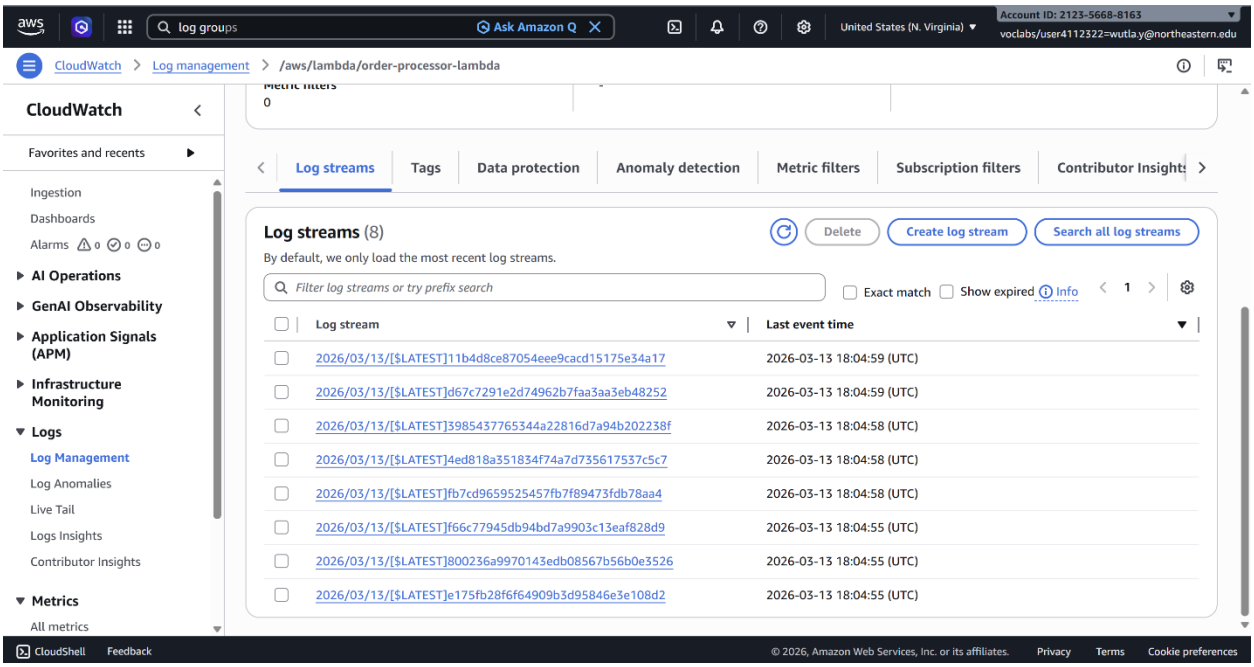
Part II: Order API → SNS → SQS → ECS Workers (manually managed)

Part III: Order API → SNS → Lambda (AWS managed)



Cold Start Observations

8 orders were sent through the existing async endpoint and processed by Lambda.



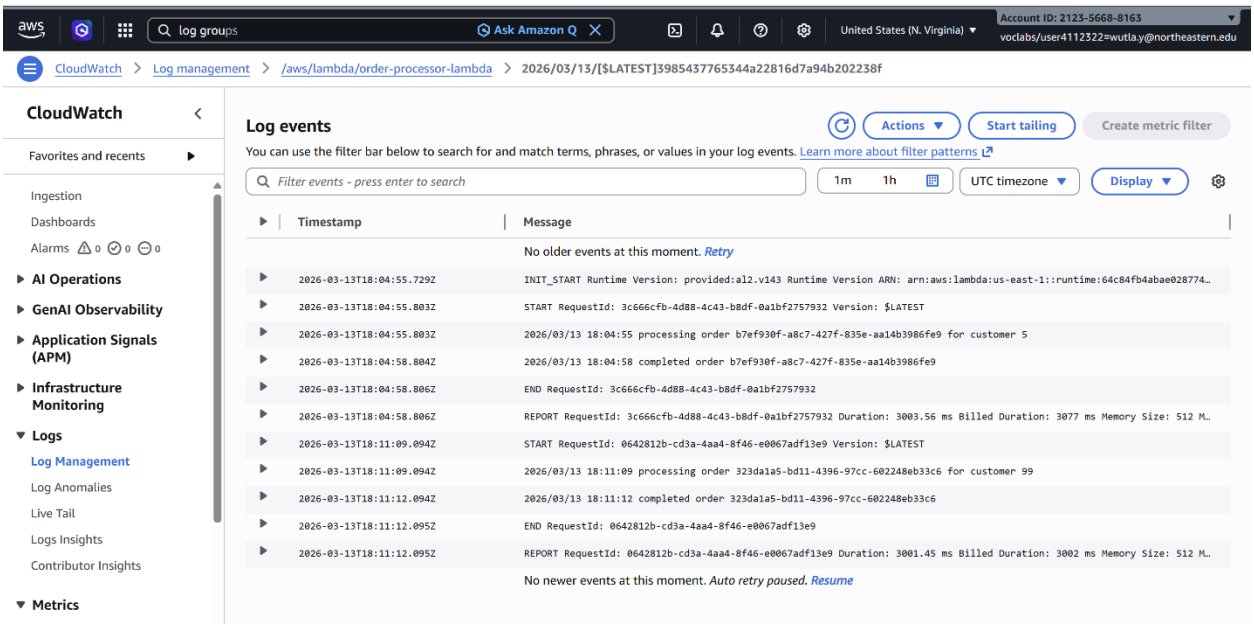
All 8 initial invocations were cold starts:

Invocation	Init Duration	Processing Duration
Order 1	73.07ms	3,003ms
Order 2	72.52ms	3,003ms
Order 3	75.26ms	3,006ms
Order 4	59.50ms	3,002ms
Order 5	72.96ms	3,003ms
Order 6	104.64ms	3,003ms
Order 7	78.75ms	3,005ms
Order 8	74.13ms	3,042ms
Average	**76.35ms**	**~3,008ms**

Cold start overhead = ~2.5% of total execution time.

Warm Start Observation

After ~6 minutes of idle time, one additional order was sent:



REPORT Duration: 3001.45ms Billed Duration: 3002ms Memory: 512MB Max Used: 20MB

No Init Duration field ,Lambda reused an existing warm instance. Billed duration nearly equals actual duration, confirming zero cold start overhead.

Cold starts occur: on first invocation or after ~5+ minutes of inactivity. For a 3-second payment processing function, the 76ms average cold start overhead is negligible (2.5% impact).

Cost Analysis

Current ECS cost (Part II): 2 ECS tasks × $8.50/month = **$17/month** (always running regardless of load)

Lambda cost:

AWS Lambda pricing:

- $0.20 per 1 million requests
- $0.0000166667 per GB-second
- Free tier: 1M requests + 400,000 GB-seconds monthly

For our workload (3 seconds, 512MB = 0.5GB):

- GB-seconds per order = 3 × 0.5 = 1.5 GB-seconds/order
- Free tier covers: 400,000 ÷ 1.5 = **~267,000 orders/month FREE**
- Break-even with ECS: ~680,000 orders/month before Lambda costs $17

Lambda is FREE for any startup processing under 267,000 orders/month.

Switch Recommendation

Based on our observations, a startup at early stage should switch to Lambda for order processing. Cold starts occurred only on the first invocation or after several minutes of inactivity, adding 60–105ms of overhead on top of a 3-second payment processing operation ,a 2–3% impact that customers would never notice. More importantly, Lambda eliminated all operational overhead: no SQS queue depth monitoring, no manual goroutine tuning, no ECS health checks, and no always-on cost. AWS automatically scaled to 8 concurrent Lambda instances when 8 simultaneous orders were sent, something that required explicit goroutine configuration with ECS workers. The cost advantage is decisive , Lambda processes up to 267,000 orders per month completely free under the AWS free tier, compared to $17/month for always-running ECS tasks. The trade-off is losing SQS guarantees (SNS retries only twice before discarding), but for a startup where operational simplicity matters more than enterprise-grade retry semantics, Lambda is the clear choice until order volume exceeds several hundred thousand per month.